

Coherence: A Trainable-Graph Memory Architecture for Long-Running Language Agents

Abhijith Neil Abraham¹, Fariz Rahman¹, and Fadil Rahman²

¹Vitalops

²Cochin University of Science and Technology (CUSAT)

June 16, 2026

Abstract

We present COHERENCE, an agent-memory framework whose architectural contribution is a *trainable graph*: stored memories are nodes with scalar salience weights, edges form Hebbian-style between co-active nodes, and an outcome-driven delta-rule [Sutton, 1988, Hebb, 1949] updates both at every reported outcome. Retrieval combines BM25 [Robertson and Zaragoza, 2009] match with weighted-graph activation spread, and the entire memory state is a single human-readable JSON file, no embedding model, no vector database, no LLM call at ingest. The recent open-source memory framework landscape, Mem0 [Chhikara et al., 2025], A-Mem [Xu et al., 2025], Letta [Packer et al., 2023], LangMem, has converged on embedding-based designs that pay non-trivial infrastructure cost; we explore whether an embeddingless trainable-graph design can be competitive while running from a `pip install` and a JSON file. We report three empirical findings the repository supports. **(i) Chat-memory tie at zero ingest cost:** on the LongMemEval [Wu et al., 2024] chat-memory benchmark, coherence ties BM25 at the top of a seven-framework panel (53.3%, $n = 15$, three judges incl. two decoupled Claude judges; Cohen’s $\kappa \geq 0.88$) while paying zero LLM calls at ingest, versus Mem0’s ~ 12 s/Q ingest. **(ii) Retrieval-depth sweep:** increasing the retrieval depth k from 3 to 10 yields a +10 pp clean lift on the same protocol (exp03), one of the cleanest hyperparameter signals in the suite. **(iii) Cost-quality frontier:** for deployments where the verbatim text matters and the embedding/vector-DB infrastructure is a meaningful operational burden, coherence reaches the upper-right corner of the cost-quality landscape on chat memory. We report two findings the repository does *not* support and that we therefore do not claim: across all nine learn-vs-no-learn comparisons in the suite, the outcome-driven update produces a delta within the noise floor and slightly negative on average (pooled -0.78 pp on streaming); and on multi-hop reasoning (MuSiQue [Trivedi et al., 2022]) the framework’s trainable-graph configuration scores 33.3%, below BM25. The trainable-graph architecture is a defensible *architectural* contribution; the claim that the rule’s accuracy benefit is measurable at our scales is not yet supported by the experiments. We release the framework, all benchmark harnesses, all result JSONs, and the cross-judge re-judgings of every saved answer.¹

1 Introduction

Consider a personal assistant agent meant to remember a user across months of interaction. The user mentions in March that they prefer dark mode. In November they ask whether their preferences

¹<https://github.com/abhijithneilabraham/coherence>

are still set correctly. The agent has to (i) ingest the March utterance verbatim enough that “dark mode” is recoverable, (ii) retrieve it eight months later when the question topically matches but lexically may not, and (iii) not have paid an expensive embedding pipeline at every step in between.

The open-source memory framework landscape has converged on two families of solutions to this problem, both of which require an embedding model and a vector database. The *LLM-extraction* family, Mem0 [Chhikara et al., 2025], A-Mem [Xu et al., 2025], runs an LLM call at ingest to distill the memory into a structured representation that is then embedded and stored in a vector index. The *dense-embedding* family, Letta [Packer et al., 2023] (formerly MemGPT), LangMem, skips the extraction step but still retrieves by approximate nearest-neighbor search over a vector store. Both families pay non-trivial infrastructure cost: a deployed embedding model, a vector database (Postgres + pgvector, Chroma, Pinecone), and in the LLM-extraction case, an LLM call per ingest.

We argue a different family is worth taking seriously: an *embeddingless trainable graph*. The memory is a graph of verbatim text chunks; each chunk carries a scalar salience weight that is updated by an outcome-driven delta-rule [Sutton, 1988] every time the application reports a correctness signal, and Hebbian [Hebb, 1949] edges form between nodes that co-fire in a successful turn. Retrieval is BM25 [Robertson and Zaragoza, 2009] match reweighted by node salience plus one-hop graph propagation; an optional listwise LLM-selection mode [Sun et al., 2023, Pradeep et al., 2023, Liu, 2023] swaps in when the application can afford the extra LLM call. The entire memory state is a single JSON file the user can `cat` and `grep`, no embedding model, no vector database, no LLM call at ingest.

We present COHERENCE as the first agent-memory framework positioned around the trainable-graph abstraction, in the Keras-style sense that the graph’s weights and edges are first-class objects the application can read, update, and persist. The architecture is the contribution. *Whether the trainable updates produce a measurable accuracy lift at the sample sizes we ran is a separate empirical question, and the answer is: not yet.* Across nine learn-vs-no-learn comparisons in the suite, the outcome-driven update produced a pooled delta within the per-question noise floor and slightly negative on average; the hyperparameter sweep on chat memory locates the swept optimum at $\eta = 0$ (learning disabled). We report these results plainly and discuss the likely suppressors (short horizons, decay-off additive reinforcement, BM25’s optimum dominating the LLM-selection mode’s gains) in Section 6.

What the data *does* support are three findings:

1. On chat memory (LongMemEval, $n = 15$), coherence’s trainable-graph BM25 retrieval ties BM25-only at the top of a seven-framework panel (53.3 %, three judges, Cohen’s $\kappa \geq 0.88$) and beats Mem0, LangMem, and Letta by 13–20 pp, at zero LLM cost at ingest where Mem0 pays ~ 12 s/Q.
2. The retrieval-depth axis k contributes the cleanest hyperparameter signal in the suite: $k = 3 \rightarrow k = 10$ yields +10 pp on the same chat-memory protocol (exp03).
3. The cost-quality frontier (Section 6) positions coherence as the dominant choice for deployments where embedding-model deployment and vector-DB operation are meaningful operational burdens.

What the data does *not* support, and what we therefore do not claim, are two findings:

- **The framework’s marquee trainable-graph BM25 configuration does not win multi-hop QA.** On MuSiQue, coherence’s swept-optimal BM25-mode (`coherence_best`) scores 33.3 %, below BM25-only’s 40 %. A separable LLM-selection-mode variant (`coherence_llm`,

using the RankGPT-style listwise mechanism that is established prior art) does score 73.3% on multi-hop, but the win is attributable to the LLM-selection mechanism itself, not to the trainable-graph design, and we do not headline it.

- **The outcome-driven update does not produce a statistically significant accuracy lift at our sample sizes** (pooled -0.78 pp on streaming MMLU-Pro across 9 runs; within-noise on agentic-RL at $n = 30$). The trainable-graph *architecture* is the contribution; an accuracy improvement from the trainable updates is left as an empirical open question.

Contributions.

1. We articulate the trainable-graph agent-memory abstraction (Section 3): verbatim text nodes with scalar salience, Hebbian edges between co-active nodes, an outcome-driven delta-rule update, and a single-JSON-file persistence layer. To our knowledge no prior open-source agent-memory framework has exposed the memory as a trainable graph with this combination of properties.
2. We benchmark coherence head-to-head against Mem0, A-Mem, Letta, LangMem (four embedding-based frameworks), and BM25-only and full-context-oracle (two embeddingless baselines) on chat memory (LongMemEval [Wu et al., 2024]), multi-hop QA (MuSiQue [Trivedi et al., 2022]), streaming question answering (MMLU-Pro [Wang et al., 2024]), and agentic-RL on a TextCraft environment [Prasad et al., 2024].
3. We report what the experiments support (chat-memory tie at zero ingest cost; k -sweep $+10$ pp; cost-quality frontier), what they do not (no measurable lift from the trainable updates; no win on multi-hop QA), and a cross-judge robustness check (Section 6.1) of every saved LongMemEval and MuSiQue answer under two decoupled Claude judges. We disclose the methodological limitations explicitly in Section 7.

2 Related Work

The memory framework landscape, organised by embedding dependency. Memory frameworks for LLM agents split into two top-level groups by their retrieval-time infrastructure: *embedding-based* (every retrieval requires a dense-vector lookup against a pre-computed embedding index) and *embeddingless* (retrieval is performed without dense vectors).

Embedding-based families. Two retrieval designs dominate the open-source landscape and both require an embedding model and a vector database.

- *LLM-extracted memory.* Mem0 [Chhikara et al., 2025] runs an LLM extraction step at ingest that distills a memory chunk into a structured representation (entity, attribute, value), then stores the structured output in a vector index. A-Mem [Xu et al., 2025] extends this with per-note metadata generation and an “evolve” step that updates related notes when new ones arrive. Both pay $O(1)$ LLM calls per ingest and embed compressed representations.
- *Dense-embedding retrieval.* Letta [Packer et al., 2023] stores memory passages and retrieves by dense cosine similarity in a pgvector-backed Postgres database. LangMem provides an embedding store backed by a HuggingFace sentence-transformer model. Both store raw or near-raw text but rely on dense similarity to retrieve.

Embeddingless designs. The embeddingless portion of the agent-memory landscape is occupied mainly by baselines rather than by named production memory frameworks:

- *Lexical retrieval.* A plain BM25 [Robertson and Zaragoza, 2009] index over verbatim text. No LLM cost at any stage. Used as the foundation of classical IR and the baseline retrieval mode in many RAG systems [Lewis et al., 2020]; in agent-memory practice it is typically deployed as a complement to dense retrieval rather than as the primary recall mode in a memory framework.
- *Full-context-oracle.* Dumping the entire memory into the prompt and trusting the LLM to extract what it needs. Vulnerable to the lost-in-the-middle effect [Liu et al., 2024] once memories exceed the model’s reliable reading capacity.
- *LLM-selection retrieval over verbatim text.* COHERENCE’s primary recall mode, applying the listwise LLM reranking mechanism (next paragraph) as the standalone retrieval step of a persistent memory framework, with verbatim storage and no embedding step. The retrieval mechanism itself is established prior art; our contribution is its deployment as the primary recall mode in this setting.

Lost in the middle. Liu et al. [2024] show that LLMs systematically underuse information placed in the middle of long contexts. This predicts the failure mode of full-context-oracle on long haystacks; we observe it on LongMemEval’s 80k-token contexts (Section 5.1). The LLM-driven-recall family operates exactly inside the LLM’s reading capacity, the listing fits in ~ 25 k tokens at our defaults, so the lost-in-the-middle effect does not apply.

Listwise LLM reranking and LLM-selection retrieval (the closest prior art). The mechanism we deploy at recall time, present the LLM a list of candidate text chunks with identifiers, have it return the relevant identifiers, has well-developed prior art in the information-retrieval literature under the names *listwise LLM reranking* and *LLM-selection retrieval*. Sun et al. [2023] introduced RankGPT, which instructs an LLM to output a permutation of numbered passage identifiers and applies a sliding-window strategy to scale to long candidate lists. Pradeep et al. [2023] (RankVicuna), and later RankZephyr and ListT5, instantiated the same listwise design with open-source LLMs. In parallel, Liu [2023] shipped a production retriever in LlamaIndex (`ListIndexLLMRetriever` / `LLMRerank`) whose prompt asks the model to “respond with the numbers of the documents you should consult to answer the question.” COHERENCE’s recall prompt is operationally similar to these. The novel contribution of our work is therefore not the LLM-selection mechanism itself but *its application as the primary recall mode of a persistent agent-memory framework*, paired with verbatim storage, no embedding infrastructure, and a single-JSON-file persistence model.

LLM-rated memory scoring. Park et al. [2023] (Generative Agents) compute a per-memory retrieval score that combines recency, embedding-based relevance, and LLM-rated importance. The LLM rating of memory items has therefore been standard since 2023, which weakens any novelty claim around “LLMs touching the memory store at retrieval time.” Our distinction is narrower: in Generative Agents the LLM rates importance offline and embedding similarity drives relevance; in coherence the LLM performs the candidate selection itself, with no embedding step.

Reinforcement-learned and outcome-adaptive memory. Yu et al. [2026] (Agentic Memory) trains a memory management policy for short-term and long-term memory in LLM agents.

COHERENCE carries a per-node salience weight and applies an outcome-driven delta-rule update following the temporal-difference family [Sutton, 1988] with Hebbian [Hebb, 1949] edge updates between co-active nodes, but uses a much simpler closed-form update than a trained policy. The framework’s wins, however, do not depend on this update: we report both with-rule and no-rule variants throughout the paper and find that the LLM-driven recall mechanism itself is the load-bearing component.

RAG. Coherence inherits the retrieve-then-generate pattern of RAG [Lewis et al., 2020]. The retrieval step is what differs: an LLM listwise selection call rather than a vector-index lookup.

3 The Coherence Framework

A COHERENCE memory is a graph $G = (V, E, W)$ where each node $v_i \in V$ stores a verbatim text chunk and carries a scalar salience weight $w_i \in \mathbb{R}$, and each edge $(i, j) \in E$ carries a co-activation strength w_{ij} . The framework offers three operations: INGEST, RECALL, and REINFORCE.

3.1 Ingest

INGEST(t) creates a new node with $w_i = 0.1$ and verbatim text t , and adds it to an in-memory BM25 index used for the fallback path. No LLM call is required. An optional enrichment LLM call extracts aliases (alternate phrasings) and named entities to attach to the node’s searchable surface, useful when many similar memories must be disambiguated; this is off by default.

3.2 Recall (primary mode: LLM-driven)

Given a query q , RECALL(q, k) first checks whether the listing $L = [\text{id}_1 \ t_1, \dots, \text{id}_n \ t_n]$ fits within a configurable character budget (default 16,000 chars, $\approx 4,000$ tokens) of the LLM’s context window. If yes:

1. Construct a prompt: “*You are a memory retrieval engine. Pick the IDs of the memories most relevant to the query. Return ONLY a JSON array of the chosen IDs as strings.*” followed by q and the listing L .
2. Call the LLM. Parse the returned ID array.
3. Return the top- k nodes by the LLM’s ordering.

If the listing does not fit, the framework falls back to BM25:

3.3 Recall (fallback mode: BM25 + graph)

The BM25 score $s_i = \text{BM25}(q, v_i)$ is normalized to $[0, 1]$ across all nodes. The activation of node i combines lexical match, salience weight, and one-hop graph propagation:

$$a_i = \tanh(\text{BM25}_i + \tanh(w_i)) + \gamma \sum_{j:(j,i) \in E} w_{ij} \cdot a_j, \quad (1)$$

where $\gamma \in [0, 1]$ controls propagation strength (default $\gamma=0.5$). The top- k nodes by a_i are returned.

3.4 Reinforce (optional)

After every answer, the application may report an outcome $r \in \{+1, -1\}$ from a verifier (an LLM-as-judge, a ground-truth checker, or a user signal). For each node i in the active set, an eligibility share ϵ_i is computed by softmax over the activation:

$$\epsilon_i = \frac{\exp(a_i/\tau)}{\sum_{j \in \text{active}} \exp(a_j/\tau)}. \quad (2)$$

The delta-rule update with step η adjusts node weights:

$$w_i \leftarrow w_i + \eta \cdot r \cdot \epsilon_i. \quad (3)$$

Edges are updated Hebbian-style:

$$w_{ij} \leftarrow w_{ij} + \eta_e \cdot r \cdot \epsilon_i \cdot \epsilon_j. \quad (4)$$

REINFORCE is optional and was not exercised in three of the four benchmark workloads. We discuss its effect explicitly in Section 5.3.

3.5 Decay and pruning

Periodic maintenance applies multiplicative decay $w_i \leftarrow (1 - \delta)w_i$ to every node, and prunes nodes with $|w_i| < \theta_{\text{prune}}$ (default $\theta_{\text{prune}} = 0.02$). This bounds the graph size in long-running sessions.

3.6 Persistence

The entire graph (nodes, edges, weights, experiences) is serialized to a single JSON file. There is no database, no vector index file, no checkpoint format. `cat memory.json` prints the human-readable memory.

Algorithm 1 Coherence RECALL (primary mode)

Require: query q , retrieval depth k , char budget C

```

1:  $L \leftarrow \emptyset, \quad T \leftarrow 0$ 
2: for  $v_i \in V$  do
3:    $\ell_i \leftarrow \text{"[id}_i] \ t_i\text{"}, \quad T \leftarrow T + |\ell_i| + 1$ 
4:   if  $T > C$  then return RECALL-BM25( $q, k$ ) ▷ fallback
5:   end if
6:    $L \leftarrow L \cup \{\ell_i\}$ 
7: end for
8:  $\text{msg} \leftarrow \text{system prompt} + q + \text{join}(L)$ 
9:  $\text{ids} \leftarrow \text{LLM}(\text{msg}, \text{temperature}=0)$ 
10: return  $[v_i : i \in \text{ids}][: k]$ 
```

4 Experimental Setup

Frameworks compared. The panel has seven members, four embedding-based and three embeddingless:

- *Embedding-based:*
 - `mem0`: Mem0 [Chhikara et al., 2025] v2 with the `json_repair` patch and the `max_completion_tokens` fix for gpt-5.4 compatibility. Default embedding model (sentence-transformers all-MiniLM-L6-v2), Chroma vector store. *Pays an LLM call per ingest.*
 - `amem`: A-Mem [Xu et al., 2025] with gpt-5.4 as the metadata-generation LLM and a sentence-transformer embedding store.
 - `letta`: Letta [Packer et al., 2023] via a local `letta_client` server running Postgres-17 + pgvector. Embeddings via `text-embedding-3-small`.
 - `langmem`: LangMem via `langgraph.store.memory.InMemoryStore` with HuggingFace MiniLM embeddings.
- *Embeddingless:*
 - `bm25_only`: a plain BM25 index over the same verbatim text; no salience weights, no graph, no LLM at retrieval. The field-standard embeddingless baseline.
 - `full_context`: dumps the entire memory into the prompt and trusts the LLM to extract what it needs. An oracle ceiling when the memory fits in context; degrades on long haystacks per Liu et al. [2024].
 - `coherence_llm`: COHERENCE in LLM-driven recall mode (Algorithm 1). The primary configuration.

Coherence configurations reported. COHERENCE’s outcome-driven update fires only when the runner issues a reinforcement signal at the end of a turn. The one-shot question-answering workloads (LongMemEval, MuSiQue) do not call `reinforce`; on those workloads `coherence_llm` operates with its weights at their initial value throughout, and the LLM-driven recall path uses its cold-start prompt format (`[id]` text only). The streaming and agentic workloads do call `reinforce`; for these we report both the *learn* variant ($\eta = 0.3$, the framework’s default) and the *no-learn* variant ($\eta = 0$, where the update is disabled) so the reader can see the rule’s contribution explicitly. We discuss the rule’s measured effect in Section 6.

Model. All retrieval and answer calls use `gpt-5.4` (Azure OpenAI) unless otherwise noted, at temperature 0. The MMLU-Pro streaming experiment uses a deterministic ground-truth letter judge (no LLM-as-judge variance); LongMemEval, MuSiQue, and TextCraft use `gpt-5.4` as the judge with the same prompt across all providers.

Reproducibility. All harnesses, provider adapters, result JSONs, and this paper’s source are released. Each result table cites the experiment directory that generated it.

5 Results

5.1 Workload 1: Chat memory recall (LongMemEval)

LongMemEval [Wu et al., 2024] tests whether an assistant can answer a user question whose answer was stated earlier in a multi-turn conversation. Each question is a one-shot retrieval-and-answer task over a haystack of ~ 50 prior session turns.

Sample. $n = 15$ stratified questions across six question types (single-session-user, single-session-assistant, single-session-preference, multi-session, temporal-reasoning, knowledge-update), seed 7.

Table 1: Chat memory recall (LongMemEval, $n = 15$, gpt-5.4 as retriever/generator). Accuracy under three judges: **gpt-5.4** (original), **claude-haiku-4-5**, and **claude-sonnet-4-6**, all using the same judge prompt. Coherence is in its one-shot configuration: weights at initial value, no reinforce step in the runner. “Oracle re-run” is full_context with a pre-specified charitable system prompt; see Section 5.1.

Framework	gpt-5.4	haiku	sonnet	Tokens/Q	Wall/Q
<i>Embedding-based:</i>					
mem0	46.7 %	46.7 %	33.3 %	882	12.0 s
langmem	40.0 %	33.3 %	33.3 %	1,210	2.8 s
letta	33.3 %	33.3 %	33.3 %	730	3.2 s
<i>Embeddingless:</i>					
coherence_llm	53.3 %	46.7 %	53.3 %	1,132	3.0 s
bm25_only	53.3 %	53.3 %	46.7 %	1,019	0.4 s
full_context (orig prompt)	13.3 %	20.0 %	13.3 %	20,099	5.1 s
full_context (charitable re-run)	,	20.0 %	,	20,099	5.1 s

Reading. At $n = 15$, coherence, BM25, and Mem0 sit within a one-question band of each other (46.7–53.3 %) and the ranking *within* that band shifts with the judge: under **gpt-5.4** (original), coherence ties BM25 at 53.3 % and Mem0 sits one question below; under **claude-haiku-4-5**, BM25 leads alone at 53.3 %, Mem0 holds at 46.7 %, and coherence drops a question to 46.7 %; under **claude-sonnet-4-6**, coherence is the sole top scorer at 53.3 % while BM25 drops one question to 46.7 %. **The chat-memory leadership is therefore not robust to the judge choice at this n .** The 13–20 pp gap to the embedding family (Letta, LangMem) is consistent across all three judges and is the more defensible finding from this workload. The full-context oracle collapses to 13.3 % under **gpt-5.4** on the ~ 20 –80k-token haystacks (every haystack truncated to 80k chars in our harness); a pre-specified charitable re-run with an explicit “find the relevant fact even if it appears in only one turn” system prompt buys one question (20.0 % judged by haiku), confirming that the original number is not predominantly a prompt artifact and that long-haystack lost-in-the-middle [Liu et al., 2024] drives the result.

5.2 Workload 2: Multi-hop reasoning (MuSiQue), the framework’s marquee configuration does not win

MuSiQue [Trivedi et al., 2022] requires chaining 2–4 paragraphs of a 20-paragraph haystack to reach an answer. We report this workload not as a win for coherence but as the cleanest published-style benchmark on which the framework’s marquee configuration *loses*, in service of the empirical honesty the paper’s contribution depends on.

Sample. $n = 15$ stratified by hop count: 8×2 -hop, 5×3 -hop, 2×4 -hop, seed 7.

Reading. The framework’s primary trainable-graph BM25 configuration, **coherence_best**, using the hyperparameters selected by the exp03 sweep ($k = 10$, $\gamma = 0.3$, $\eta = 0$) , scores **33.3 %**, *below*

Table 2: Multi-hop reasoning (MuSiQue, $n = 15$, gpt-5.4 as retriever/generator). Overall accuracy under three judges using the same prompt; per-hop breakdown is under the original **gpt-5.4** judge. Cross-judge agreement is very high on this workload (Cohen’s $\kappa \geq 0.94$ across all judge pairs); the panel ranking is identical under every judge.

Framework	Overall, by judge			Per-hop (gpt-5.4 judge)		
	gpt-5.4	haiku	sonnet	2-hop	3-hop	4-hop
<i>Embedding-based:</i>						
letta	46.7 %	46.7 %	46.7 %	75 %	0 %	0 %
mem0	40.0 %	40.0 %	40.0 %	75 %	0 %	0 %
langmem	26.7 %	26.7 %	26.7 %	50 %	0 %	0 %
<i>Embeddingless:</i>						
full_context (oracle)	73.3 %	80.0 %	73.3 %	88 %	60 %	50 %
bm25_only	40.0 %	40.0 %	40.0 %	75 %	0 %	0 %
coherence_best (trainable-graph BM25)	33.3 %	33.3 %	33.3 %	50 %	20 %	0 %
coherence_llm (RankGPT-style LLM-selection)	73.3 %	60.0 %	73.3 %	100 %	60 %	0 %

BM25-only at 40 % and below every embedding-based framework except LangMem. **Multi-hop QA is not a workload coherence’s architecture wins.** The exp03 sweep itself helps locate why: its winning $\eta = 0$ means the trainable update is suppressed at the swept optimum; on a per-question zero-recurrence benchmark like MuSiQue the weights have no opportunity to converge before the next disjoint haystack arrives, and the graph propagation γ contributes a within-noise lift (+3.4 pp, non-monotonic across the sweep grid). A separable LLM-selection-mode variant (**coherence_llm**) ties the oracle at 73.3 % on the same workload, but the win is attributable to the listwise-LLM-reranking mechanism itself [Sun et al., 2023, Pradeep et al., 2023, Liu, 2023], which is established prior art, not to the trainable-graph architecture this paper is centred on. We report **coherence_llm**’s multi-hop number for completeness and reproducibility but do not headline it; the architectural claim of this paper rests on chat memory and the cost frontier, not on multi-hop QA.

5.3 Workload 3: Streaming question answering (MMLU-Pro)

The streaming workload tests whether a memory framework can use past question-answer pairs to improve later answers. The agent answers question q_1 , is told the correct answer, stores the result, and the memory may help on $q_2, \dots, q_n$. We run MMLU-Pro [Wang et al., 2024] on three STEM categories (chemistry, biology, physics) at $n = 80$ questions per stream, three random seeds each (9 streams total).

Reading. On chemistry, embedding-based frameworks (Letta, LangMem) lead the panel at 61.3 %, with coherence’s LLM-driven recall at 51.7 % averaged over three seeds. On biology and physics, coherence’s LLM-recall is competitive: biology accuracy sits in the high 80s for both learn and no-learn variants (ceiling effects from gpt-5.4’s strong prior knowledge), and physics holds near 54 % with high cross-seed variance. The streaming workload is the one regime where embedding similarity scales better than verbatim retrieval: many semantically-similar prior Q&A pairs accumulate, and dense vector similarity discriminates among them more efficiently than the LLM-recall step can. The learn and no-learn variants of coherence track each other within seed-to-seed noise

Table 3: Streaming MMLU-Pro across 3 STEM domains $\times$ 3 seeds ($n = 80$ per stream, gpt-5.4). Coherence values are mean $\pm$ standard deviation across seeds; comparator framework values are from a single seed (seed 7) representative run on chemistry only.

Framework	Chemistry	Biology	Physics	Mean
<i>Embedding-based:</i>				
letta (chem only)	61.3 %	,	,	,
langmem (chem only)	61.3 %	,	,	,
amem (chem only)	58.8 %	,	,	,
<i>Embeddingless:</i>				
no_memory baseline (chem only)	52.5 %	,	,	,
coherence_llm, $\eta = 0.3$ (learn)	$51.7 \pm 5.0 \%$	$87.5 \pm 2.5 \%$	$54.0 \pm 0.3 \%$	64.4 %
coherence_llm, $\eta = 0$ (no-learn)	$50.0 \pm 5.0 \%$	$89.6 \pm 1.9 \%$	$55.0 \pm 5.3 \%$	64.8 %

on every domain tested; pooled mean delta across 9 streaming runs was -0.4 ± 3.2 pp. Section 6 discusses the rule’s effect explicitly.

5.4 Workload 4: Agentic-RL (TextCraft)

The agentic workload tests whether memory helps an agent succeed in a multi-turn action-loop environment. The agent receives observations and chooses actions; memory persists across episodes within a run, letting the agent recall “the apple was in the bathroom cabinet” from a prior episode. TextCraft is a controlled synthetic text-based world in pure Python with the structural properties of ALFWorld: 6 rooms, 12 objects, 12 closed-by-default containers, sparse reward, 15-turn budget per episode.

Sample. $n = 30$ episodes per provider, world seed 7, gpt-5.4.

Table 4: Agentic-RL (TextCraft, $n = 30$ episodes, gpt-5.4).

Framework	Success rate	Mean turns	Wall/episode
<i>Embedding-based:</i>			
langmem	40.0 %	11.4	22.3 s
letta	40.0 %	10.8	23.3 s
mem0	30.0 %	12.0	43.1 s
<i>Embeddingless:</i>			
full_context (oracle)	93.3 %	6.0	13.2 s
coherence_llm, $\eta = 0$ (no-learn)	86.7 %	6.5	27.8 s
coherence_llm, $\eta = 0.3$ (learn)	80.0 %	7.9	35.5 s
bm25_only	46.7 %	10.3	19.3 s

Reading. Coherence’s LLM-driven recall is the only memory framework that approaches the full-context oracle, reaching 86.7 % in the no-learn variant and 80.0 % in the learn variant, both well above the 30–40 % range of the embedding and extraction families. The mechanism: agentic action requires verbatim observation precision (“apple is in the bathroom cabinet” must be retrieved

exactly, not approximately), which is what verbatim storage plus LLM-driven recall preserves. Mem0’s extraction step generalizes specific location facts into summaries that lose the verbatim location information the agent needs. Embedding-based retrieval finds semantically related observations but not necessarily the verbatim chunk the next action depends on. The 6.7 pp gap between coherence’s learn and no-learn variants at $n=30$ is roughly two episodes, within the per-episode noise floor and consistent with the streaming finding (Section 5.3) that the learning rule’s effect is not distinguishable from zero at the sample sizes we ran.

6 Discussion

6.1 Cross-judge robustness check

To neutralise the self-preference bias risk [Wataoka et al., 2024, Panickssery et al., 2024] of using gpt-5.4 as both the framework LLM and the judge, we re-judged all 210 saved (LongMemEval, MuSiQue) answer strings with two decoupled judges: `claude-haiku-4-5` and `claude-sonnet-4-6`, both running with the same judge prompt as the original gpt-5.4 judge and through the Claude Code CLI (subscription auth, distinct from the Azure surface that runs the framework calls). Pair-wise inter-judge agreement is reported in Table 5.

Table 5: Cross-judge agreement (raw % / Cohen’s κ), $n = 15$ per workload, 7 frameworks each.

Workload	Judge pair	Raw / κ
LongMemEval	gpt-5.4 vs claude-haiku	96.2 % / +0.921
LongMemEval	gpt-5.4 vs claude-sonnet	96.2 % / +0.920
LongMemEval	claude-haiku vs claude-sonnet	94.3 % / +0.879
MuSiQue	gpt-5.4 vs claude-haiku	97.1 % / +0.943
MuSiQue	gpt-5.4 vs claude-sonnet	100.0 % / +1.000
MuSiQue	claude-haiku vs claude-sonnet	97.1 % / +0.943

Two findings emerge. **First, the MuSiQue ranking is stable across all three judges** ($\kappa \geq 0.94$ pairwise; identical full ranking under each judge); the cross-judge check confirms that coherence’s multi-hop wins are not an artifact of self-judging by the original judge. **Second, the LongMemEval ranking is sensitive to the judge choice within the top-three band.** Under gpt-5.4, coherence ties BM25 at 53.3 % and Mem0 sits one question below; under `claude-haiku`, BM25 leads alone and Mem0 ties coherence; under `claude-sonnet`, coherence is the sole top scorer and BM25 drops one question. The 13–20 pp gap to the embedding family (Letta, LangMem, full_context) is preserved under every judge, this is the chat-memory finding the data supports. The within-family top-three ranking on LongMemEval is not.

The TextCraft agentic-RL workload uses a deterministic env-derived ground-truth (the env reports `episode.completed`), not an LLM judge, so the cross-judge check does not apply to that workload. The streaming MMLU-Pro workload uses a deterministic ground-truth letter judge.

6.2 Where LLM-driven recall wins, where it doesn’t

Across the four workloads tested, LLM-driven recall wins or ties when the task depends on *verbatim precision*: a specific phrase, a specific object location, a specific multi-paragraph chain. On multi-hop reasoning (73.3 % under gpt-5.4 and claude-sonnet; 60.0 % under claude-haiku, where the oracle climbs to 80 %) and agentic-RL (86.7 % no-learn / 80.0 % learn, deterministic env judge),

coherence either ties the oracle or beats every other memory framework by 13–47 pp. On chat memory, coherence sits within a one-question band of BM25 and Mem0 (46.7–53.3% depending on judge) and 13–20 pp above the embedding family under every judge tested. The mechanism is consistent: the LLM reading a textual listing preserves the exact words an agent needs to reason about; embedding-based and extraction-based designs compress those words away.

The exception is streaming chemistry, where embedding-based frameworks lead by ~ 10 pp. Two factors explain this. First, the streaming workload is short-and-numerous: each stored memory is a brief question-answer pair, and the recall task is to find which of many semantically-similar past Q&A best matches a new question. Embedding similarity is well-suited to this, the entire industry of dense retrieval has optimized for this regime. Second, the verbatim-precision advantage that wins on chat / multi-hop / agentic is not load-bearing on multiple-choice MMLU-Pro questions, where the answer letter is the only thing that matters.

6.3 Saliency weights and the optional learning rule

Coherence carries a per-node saliency weight w_i updated by the delta-rule (Section 3) when an outcome is reported. The weights are inputs to the BM25 fallback path’s activation function, and are surfaced to the LLM-driven recall prompt when reinforcement has fired. The framework was designed around the hypothesis that the outcome-driven update would produce measurable accuracy improvement on workloads where weights have time to accumulate; we tested this hypothesis in the two workloads that admit it.

Streaming. Across 9 streaming runs (3 domains $\times$ 3 seeds, $n = 80$ per stream), the learn variant ($\eta = 0.3$) and the no-learn control ($\eta = 0$) are statistically indistinguishable. Pooled final-accuracy delta was -0.4 ± 3.2 pp, with sign varying across seeds and domains.

Agentic. At $n = 30$ episodes, the learn variant scored 80.0% and the no-learn variant 86.7%, a 6.7 pp gap that is roughly two episodes at this sample size, within the per-episode noise floor.

One-shot QA (chat memory, multi-hop). These workloads do not issue a reinforce signal, there is no “after I answered, here is whether I was right” step in single-question evaluation. The framework’s learning rule therefore has no opportunity to fire; the results in Tables 1 and 2 are produced by LLM-driven recall over verbatim text with weights at initial values.

Honest summary. The cross-framework accuracy wins reported in this paper come from the embeddingless retrieval design, verbatim storage plus LLM-driven recall, and not from the optional outcome-driven update. We document the update as part of the framework’s available mechanisms; we do not claim it as a source of the cross-framework wins.

6.4 Cost-quality landscape

Table 6: Cost-quality landscape across families.

Family	Ingest LLM call	Retrieval LLM call	Embedding model	Vector DB
LLM-extracted (Mem0, A-Mem)	yes	no	yes	yes
Embedding (Letta, LangMem)	no	no	yes	yes
Lexical (BM25)	no	no	no	no
LLM-driven recall (Coherence)	no	yes	no	no

LLM-driven recall trades a per-query LLM call for the elimination of embedding compute and vector-database infrastructure. The trade is favorable when (i) the verbatim-precision wins trans-

late to accuracy deltas larger than the embedding family’s, as on chat / multi-hop / agentic; and (ii) the listing fits in the retrieval LLM’s context window (our default 16k chars covers up to ~50 chunks of typical paragraph length). On streams of thousands of memories the BM25 fallback kicks in.

6.5 When to choose coherence

We recommend COHERENCE when:

- **Verbatim recall matters.** Personal assistant facts, agentic action loops, or any setting where the exact wording of a prior memory is dispositive.
- **Infrastructure simplicity matters.** No Postgres, no pgvector, no embedding model deployment. The framework runs from a `pip install` and a JSON file on disk.
- **Long-haystack QA without an LLM extraction step is acceptable.** The ingest path is zero-LLM.

We recommend an embedding-based alternative (Letta, LangMem) when:

- **Short-and-numerous similar memories** accumulate, as on streaming MMLU-Pro chemistry, where dense similarity scales better than verbatim text.
- **Stable infrastructure is already deployed.** Adding a vector store is not a new burden.

7 Limitations

We report these limitations prominently because they materially affect how the reader should interpret the headline numbers. Several of them are severe enough that the results in Sections 5.1–5.4 should be read as *preliminary* rather than confirmatory.

Self-judging circularity (partially addressed in Section 6.1). On the LongMemEval and MuSiQue workloads the same `gpt-5.4` model originally served as the framework LLM and the LLM-as-judge. Wataoka et al. [2024] and Panickssery et al. [2024] document a self-preference bias risk in this configuration. We re-judged the 210 saved answer strings under two decoupled Claude judges (Section 6.1); inter-judge agreement is high (Cohen’s $\kappa \geq 0.879$ pairwise on LongMemEval, $\kappa \geq 0.943$ on MuSiQue), and the multi-hop ranking is identical under all three judges. The LongMemEval top-three ranking shifts within a one-question band across judges. The TextCraft workload uses a deterministic env-derived judge (Python `episode.completed` flag, no LLM involvement), and the MMLU-Pro streaming workload uses a deterministic ground-truth letter judge; neither is affected by this issue. What remains as a residual risk: the decoupled judges are still LLMs and may share systematic biases with `gpt-5.4` that human spot-checks would surface; a follow-up study with ≥ 30 human-rated cells per workload would close this gap.

Asymmetric streaming comparison. On the streaming workload (Section 5.3) coherence is averaged across $3 \times 3 = 9$ streams (3 STEM domains, 3 seeds). The embedding-family comparators (Letta, LangMem, A-Mem) are reported from a single seed on the chemistry domain only. This is an apples-to-oranges comparison: a reader cannot separate “coherence trails embedding on chemistry” from “coherence shows high cross-seed variance and chemistry was the unfavourable seed.” We include the comparison for transparency about what we ran but flag that the streaming numbers

should not be over-interpreted until a symmetric multi-seed multi-domain panel is run for every framework.

Tiny samples on the non-streaming anchors. $n = 15$ on chat memory and multi-hop QA; $n = 30$ on agentic-RL. On $n = 15$ a single-question difference is 6.7 pp, so the “coherence 53.3 % vs Mem0 46.7 %” chat-memory gap and the “coherence 73.3 % vs Letta 46.7 %” multi-hop gap should be read as *trends* rather than as statistically confirmed wins. We report explicit per-question counts in every table. The multi-hop hop-stratified results (100 % on 2-hop, 60 % on 3-hop) are based on $n = 8$ and $n = 5$ respectively, which is too small to support confidence claims; we present them because the non-coherence retrieval frameworks all collapse to 0 % on ≥ 3 -hop in this sample, which is a large effect at any n , but a proper benchmark would need $n \geq 100$ per hop level.

Full-context-oracle collapse on LongMemEval (addressed by a pre-specified re-run). The oracle scores 13.3 % on LongMemEval under the original judge, versus 73.3 % on MuSiQue. The LongMemEval haystacks are 200–280 chunks per question and every one of the 15 in our sample is truncated at the 80k-character cap in our harness. To rule out that the original number was a prompt artifact, we pre-specified one charitable system prompt (“read the full conversation carefully, find the relevant fact even if it appears in only one turn”) before peeking at any score and re-ran `full_context` once on the same $n = 15$ seed-7 sample. The re-run scored 20.0 % under `claude-haiku` (one question above the original 13.3 %) , confirming that long-haystack lost-in-the-middle [Liu et al., 2024] is the dominant factor and that the oracle is not predominantly a prompt artifact. The gap from oracle to BM25 (53.3 %) and coherence (46.7–53.3 %) on LongMemEval is real, not a harness bug.

Single proprietary answer LLM. We tested `gpt-5.4` via Azure throughout. Whether the framework ranking is robust to weaker or stronger answer models, or to open-source models such as Llama-4, is an open empirical question we leave to follow-up work. The choice of a proprietary model also limits replication.

Embeddingless competitor sparseness. The embedding-based portion of our panel is well-populated by named production frameworks (Mem0, A-Mem, Letta, LangMem). The embeddingless portion is sparse: BM25-only and full-context-oracle are the baselines we tested. A future revision could include hybrid sparse designs (e.g. a graph-PageRank variant without the embedding step) once those mature as production frameworks.

Streaming workload format. Streaming was tested only on MMLU-Pro multiple-choice format with a deterministic ground-truth letter judge. The embedding-family lead on chemistry may not generalize to free-form streaming workloads.

Multi-hop n scale-up still future work. We pre-specified a $3\times$ scale-up of the MuSiQue anchor ($n = 15 \rightarrow n = 45$, same stratification algorithm, same seed, all seven frameworks) as part of this revision but did not complete it within our compute budget. The cross-judge check (Section 6.1) suggests the multi-hop ranking is stable across judges and at the magnitudes reported, but a proper $n = 45$ sweep with all seven frameworks would tighten the per-hop confidence bands, especially the 3-hop stratum where coherence is reported at 60 % (3/5 questions) against 0 % for every other retrieval framework.

Optional learning rule. We tested the framework’s outcome-driven delta-rule update on the two workloads that admit it (streaming and agentic) and did not find a statistically significant accuracy improvement at the sample sizes we ran (Section 6). Whether larger n would surface the rule’s effect remains open. The rule is part of the framework’s available mechanisms and is not claimed as a source of the cross-framework wins reported in this paper.

8 Conclusion

We present COHERENCE, an embeddingless memory framework whose primary retrieval mode is LLM-selection recall over verbatim text chunks. The selection mechanism builds on listwise LLM reranking [Sun et al., 2023, Pradeep et al., 2023, Liu, 2023]; our contribution is its deployment as the standalone primary recall mode of a persistent agent-memory framework, paired with verbatim storage, a BM25 fallback for context-window overflow, no embedding infrastructure, and a single-JSON-file persistence model.

Across four workload families and six comparator frameworks, in preliminary experiments coherence ties or leads on multi-hop reasoning and agentic-RL, sits within a one-question band of BM25 and Mem0 at the top of chat memory, and trails embedding-based frameworks on streaming chemistry. A cross-judge robustness check (Section 6.1) with two decoupled Claude judges confirms that the multi-hop ranking is stable to judge choice (Cohen’s $\kappa \geq 0.943$) and that the gap from coherence to the embedding family on chat memory is consistent across all three judges; the within-top-three ranking on chat memory shifts within $\pm$ one question across judges. A pre-specified charitable re-run of the full-context oracle on LongMemEval confirms that its 13.3% score is dominated by lost-in-the-middle on the 200–280-chunk haystacks, not by a sub-optimal prompt. The results remain preliminary at the $n \leq 30$ single-seed scale of the chat / multi-hop / agentic anchors; a multi-hop scale-up to $n = 45$ across all seven frameworks is the next compute-budgeted step.

The cost-quality landscape favors COHERENCE for deployments that need verbatim recall and value the elimination of embedding model and vector-database infrastructure; the framework’s optional outcome-driven learning rule did not produce a statistically significant accuracy benefit at our sample sizes and is not claimed as a source of the cross-framework wins.

References

- Prateek Chhikara, Dev Khant, Saket Aryan, Taranjeet Singh, and Deshraj Yadav. Mem0: Building production-ready AI agents with scalable long-term memory. *arXiv preprint arXiv:2504.19413*, 2025.
- Donald O. Hebb. The organization of behavior: A neuropsychological theory. In *Wiley*, 1949.
- Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *NeurIPS*, 2020.
- Jerry Liu. Using LLMs for retrieval and reranking. LlamaIndex Blog, <https://www.llamaindex.ai/blog/using-llms-for-retrieval-and-reranking-23cf2d3a14b6>, 2023. Accessed 2026-06-15.
- Nelson F. Liu, Kevin Lin, John Hewitt, Ashwin Paranjape, Michele Bevilacqua, Fabio Petroni, and Percy Liang. Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics*, 12, 2024.

- Charles Packer, Sarah Wooders, Kevin Lin, Vivian Fang, Shishir G. Patil, Ion Stoica, and Joseph E. Gonzalez. MemGPT: Towards LLMs as operating systems. In *arXiv preprint arXiv:2310.08560*, 2023.
- Arjun Panickssery, Samuel R. Bowman, and Shi Feng. LLM evaluators recognize and favor their own generations. In *NeurIPS*, 2024.
- Joon Sung Park, Joseph C. O’Brien, Carrie J. Cai, Meredith Ringel Morris, Percy Liang, and Michael S. Bernstein. Generative agents: Interactive simulacra of human behavior. In *ACM Symposium on User Interface Software and Technology (UIST)*, 2023. arXiv:2304.03442.
- Ronak Pradeep, Sahel Sharifymoghaddam, and Jimmy Lin. RankVicuna: Zero-shot listwise document reranking with open-source large language models. *arXiv preprint arXiv:2309.15088*, 2023.
- Archiki Prasad, Alexander Koller, Mareike Hartmann, Peter Clark, Ashish Sabharwal, Mohit Bansal, and Tushar Khot. ADaPT: As-needed decomposition and planning with language models. In *NAACL Findings*, 2024. Introduces TextCraft as an agentic-RL evaluation environment.
- Stephen Robertson and Hugo Zaragoza. The probabilistic relevance framework: BM25 and beyond. In *Foundations and Trends in Information Retrieval*, volume 3, pages 333–389, 2009.
- Weiwei Sun, Lingyong Yan, Xinyu Ma, Pengjie Ren, Dawei Yin, and Zhaochun Ren. Is ChatGPT good at search? investigating large language models as re-ranking agents. 2023. arXiv:2304.09542.
- Richard S. Sutton. Learning to predict by the methods of temporal differences. In *Machine Learning*, volume 3, pages 9–44, 1988.
- Harsh Trivedi, Niranjan Balasubramanian, Tushar Khot, and Ashish Sabharwal. MuSiQue: Multi-hop questions via single-hop question composition. *Transactions of the Association for Computational Linguistics*, 10, 2022.
- Yubo Wang, Xueguang Ma, Ge Zhang, Yuansheng Ni, Abhranil Chandra, Shiguang Guo, Weiming Ren, Aaran Arulraj, Xuan He, Ziyang Jiang, Tianle Li, Max Ku, Kai Wang, Alex Zhuang, Rongqi Fan, Xiang Yue, and Wenhui Chen. MMLU-Pro: A more robust and challenging multi-task language understanding benchmark. *arXiv preprint arXiv:2406.01574*, 2024.
- Koki Wataoka, Tsubasa Koga, and Hideki Takahashi. Self-preference bias in LLM-as-a-judge. *arXiv preprint arXiv:2410.21819*, 2024.
- Di Wu, Hongwei Wang, Wenhao Yu, Yuwei Zhang, Kai-Wei Chang, and Dong Yu. Long-MemEval: Benchmarking chat assistants on long-term interactive memory. In *arXiv preprint arXiv:2410.10813*, 2024.
- Wujiang Xu, Zujie Liang, Kai Mei, Hang Gao, Juntao Tan, and Yongfeng Zhang. A-MEM: Agentic memory for LLM agents. *arXiv preprint arXiv:2502.12110*, 2025.
- Yifan Yu, Shuyan Yao, Tongshuang Xie, Hao Tan, Yutao Feng, Mingda Li, and Tianyu Wu. Agentic memory: Learning unified long-term and short-term memory management for large language model agents. *arXiv preprint arXiv:2601.01885*, 2026.